

1. Harmless Trojan simulation

Create a file called:

- suspicious_code.txt

Put this inside:

```
REM This is a harmless Trojan simulation.  
REM It pretends to be a game but only displays a fake warning.
```

```
@echo off  
echo =====  
echo    SUPER GAME v1.0  
echo =====  
echo.  
echo Starting game...  
timeout /t 2 >nul  
echo.  
echo [SIMULATION] You have been hacked!  
echo This is only a cybersecurity lab demonstration.  
pause
```

For a cleaner demonstration, save the same content as trojan_sim.bat and run it inside your VM.

Result: It only displays text. It does not modify files, steal information, establish persistence, or communicate over the network.

2. Worm simulator — maximum 3 copies

Create:

- worm_simulator.py

Use this safe version:

```
from pathlib import Path  
import shutil
```

```

MAX_COPIES = 3
current_file = Path(__file__).resolve()
folder = current_file.parent

# Find existing copies
copies = sorted(folder.glob("worm_copy*.py"))

if len(copies) >= MAX_COPIES:
    print("Maximum of 3 copies already exists.")
else:
    next_number = len(copies) + 1
    destination = folder / f"worm_copy{next_number}.py"

    shutil.copy2(current_file, destination)

    print(f"Created: {destination.name}")
    print(f"Copies created: {next_number}/{MAX_COPIES}")

```

Your folder will eventually look like:

- worm_simulator.py
- worm_copy1.py
- worm_copy2.py
- worm_copy3.py

Running the original repeatedly will stop after three copies.

Test

- python worm_simulator.py
- python worm_simulator.py
- python worm_simulator.py
- python worm_simulator.py

3. Virus infection simulation

Create a dedicated folder:

- virus_lab

Put a few harmless text files inside:

- file1.txt
- file2.txt
- notes.txt

For example:

- This is a laboratory test file.

Create:

- virus_sim.bat

with:

```
@echo off
```

```
set "TARGET=%~dp0virus_lab"
```

```
echo Starting harmless virus simulation...
```

```
echo.
```

```
for %%F in ("%TARGET%\*.txt") do (  
    echo [SIMULATION] This file was processed by virus_sim >>  
    "%%F"  
    echo Modified: %%~nxF  
)
```

```
echo.
```

```
echo Simulation completed.
```

```
pause
```

Run it

From the folder containing the BAT file:

- .\virus_sim.bat

Each .txt file in virus_lab will receive:

- [SIMULATION] This file was processed by virus_sim

4. Real-world Trojans, viruses and worms

Trojans

Trojan	Platform	Spread / Impact
Zeus	Windows	Distributed through phishing, malicious downloads and drive-by attacks; commonly targeted banking credentials.
Emotet	Windows	Frequently spread through malicious email attachments or links and was used to deliver additional malware.
Android.BankBot	Android	Banking-focused Android malware that used malicious apps to capture sensitive information and banking credentials.

Viruses

Virus	Platform	Spread / Impact
CIH (Chernobyl)	Windows	Spread through infected executable files and could damage system data and, on some systems, BIOS firmware.
Melissa	Windows/Microsoft Office	Spread through infected Microsoft Word documents and automatically emailed itself to contacts.
Michelangelo	DOS/PC	Spread through infected disks and could overwrite portions of hard-disk data when triggered.

Worms

Worm	Platform	Spread / Impact
------	----------	-----------------

WannaCry	Windows	Exploited a Windows SMB vulnerability to spread between vulnerable computers and encrypted files for ransom.
Conficker	Windows	Exploited Windows vulnerabilities and weak credentials to spread across networks and create infected botnets.
ILOVEYOU	Windows/email	Spread primarily through email messages containing a malicious attachment and then forwarded itself to contacts.

5. Fix the worm-copy detection code

The original:

```
import os

files = os.listdir('.')

for f in files:
    if 'worm_copy' in f:
        print('Worm found:', f)
```

has two problems:

- It only looks for filenames containing worm_copy.
- It doesn't identify the original worm_simulator.py.

A better solution is to explicitly recognize the simulator family.

Fixed version

```
import os

files = os.listdir('.')

for f in files:
    name = f.lower()
```

```
if (
    name == "worm_simulator.py"
    or name.startswith("worm_copy")
    and name.endswith(".py")
):
    print("Worm found:", f)
```

This detects:

- worm_simulator.py
- worm_copy1.py
- worm_copy2.py
- worm_copy3.py

Even more flexible version

If your copies could have different names, checking the content is more reliable than relying only on filenames. For this controlled lab, you can use a known marker:

```
from pathlib import Path

MARKER = "MAX_COPIES = 3"

for file in Path(".").glob("*.py"):
    try:
        content = file.read_text(encoding="utf-8")

        if MARKER in content:
            print("Possible worm simulator found:", file.name)

    except (UnicodeDecodeError, PermissionError):
        pass
```

This illustrates an important security principle: filename-based detection is weak because malware can simply rename itself.